

Coevaluación por pares

PF1822 · Especialización en Desarrollo con IA

Módulo 2 · MA04576 · Introducción a modelos generativos y consumo por API

MOMENTO

Parte A: después de entregar la actividad final. Parte B: al cerrar cada ABPRO

MODALIDAD

Parte A en parejas que asigna el tutor/a; Parte B dentro de cada equipo de ABPRO

CALIFICACIÓN

No pones nota a tu compañero/a: el tutor/a valora la calidad de tu retroalimentación

VÍNCULO CON EL PLAN

La competencia del módulo y los ABPRO de los cuatro aprendizajes esperados

Para qué sirve

Revisar el trabajo de otra persona es parte del oficio de quien desarrolla software: en un equipo real, el código pasa por revisiones antes de llegar a producción. En esta coevaluación revisas la actividad final de un compañero/a (Parte A) y valoras el trabajo en equipo de tus compañeros/as en cada ABPRO (Parte B). Tus comentarios le sirven a quien los recibe para mejorar su portafolio, y revisar te ayuda a ver tu propio trabajo con otros ojos.

Normas

- **Respeto.** Comentas el trabajo, no a la persona. Escribe como te gustaría que te escribieran.
- **Evidencia concreta.** Cada comentario apunta a algo que se puede revisar: un archivo, una línea, un commit, una fila de un CSV o una salida de `pytest`.
- **No copies soluciones.** No subas código de tu compañero/a a tu repositorio ni reescribas su trabajo: sugiere qué cambiar y por qué.
- **Claves y credenciales.** Nunca pidas, compartas ni uses claves ajenas. Para probar con la API usa tu propia clave, en tu propio `.env`. Si encuentras una clave en el repositorio de tu compañero/a, no la uses ni la copies: avísale en privado y avisa a tu tutor/a, para que tu compañero/a la revoque y cree una nueva.
- **Datos ficticios.** Trabaja solo con los insumos del curso; no agregues datos de personas o empresas reales en tus comentarios ni en tus pruebas.
- **Acuerdos.** Entrega tu revisión en la fecha que fije tu tutor/a: tu compañero/a la necesita para su portafolio.

Parte A · Revisión cruzada de la actividad final

Pasos

1. Clona el repositorio de tu compañero/a en una carpeta nueva, crea un entorno virtual e instala sus dependencias (y el modelo de spaCy, si lo usa).
2. Antes de ejecutar nada, busca claves en el código y en todo el historial. Revisa cada coincidencia: puede haber falsos positivos. Confirma también que `.gitignore` incluye `.env` y que `.env.example` no tiene valores.
3. En una terminal donde no hayas definido `OPENAI_API_KEY` ni `HF_TOKEN`, ejecuta `pytest`: todas las pruebas deben pasar sin clave y sin conexión. Anota cuántas pasan.
4. Sigue las instrucciones del README tal como están escritas. ¿Te bastaron para instalar y ejecutar el asistente?
5. Lee `prompts.md`: el mensaje de sistema, los ejemplos del few-shot, la salida en JSON, la tabla de iteraciones y las pruebas de limitaciones.
6. Abre `resultados.csv` y `consumo.csv`. Comprueba que estén las seis preguntas y recalcula una fila (por ejemplo, el ROUGE-L de Q2) con tu propio `evaluar.py`. Compara tu valor con el suyo.
7. Si quieres probar el asistente con llamadas reales, hazlo con **tu** clave y ejecútalo con Q5 y con una paráfrasis de Q3. Usa el mismo proveedor y el mismo `MODELO_EMBEDDINGS` que indica su README, porque su `faq_embeddings.json` solo sirve con ese modelo. Si el README no lo dice, anótalo como mejora.
8. Completa la tabla y el cierre, y publícalos donde indique tu tutor/a.

Comandos de referencia, para Git Bash en Windows o la terminal en macOS o Linux (en PowerShell, `grep` no existe y el entorno se activa con `.venv\Scripts\activate`):

```
git clone <enlace del repositorio de tu compañero/a> revision-par
cd revision-par
python -m venv .venv
# activa el entorno: source .venv/Scripts/activate en Git Bash (Windows), source
.venv/bin/activate en macOS o Linux
pip install -r requirements.txt
python -m spacy download es_core_news_sm
git log -p --all | grep -n -i -E "sk-|hf_|api_key|bearer "
pytest
```

Tabla de revisión

Componente de la competencia	Comprobación	Sí	En parte	No	Evidencia o comentario
Implementar la arquitectura básica de una aplicación que integre modelos de IA					

Componente de la competencia	Comprobación	Sí	En parte	No	Evidencia o comentario
	El diagrama tiene los cinco componentes y su flujo numerado coincide con lo que hace el código				
	La justificación usa escalabilidad, seguridad y mantenimiento con datos del caso, no en general				
	La clave solo se lee en el servidor (<code>asistente.py</code> o el cliente), desde una variable de entorno				
Mediante APIs					
	Hay un GET y dos POST, con tiempo de espera, y la respuesta se lee después de revisar el código de estado; 401, 429, 5xx y tiempo agotado tienen un manejo distinto y un mensaje claro				
	En <code>prompts.md</code> , el mensaje de sistema tiene rol, contexto y restricciones; los datos van delimitados; la salida JSON se valida con <code>json.loads</code> , y hay una versión zero-shot y una few-shot con dos ejemplos propios				
	<code>pytest</code> pasa sin clave ni conexión, con cinco pruebas o más del cliente				
Preprocesamiento de texto para consumo por modelos de lenguaje					
	Q6 queda sin HTML ni ruido y con <code>[correo]</code> ; "error 500" o datos parecidos se conservan				
	Las etapas son funciones separadas y la búsqueda y la evaluación usan las mismas				
	La tabla compara TF-IDF, CountVectorizer y embeddings, y el umbral se justifica con las similitudes				
Garantizando el consumo eficiente					
	<code>consumo.csv</code> registra los tokens de cada llamada, tomados de <code>usage</code>				
	Los embeddings de las preguntas frecuentes se guardan y no se recalculan en cada consulta				

Componente de la competencia	Comprobación	Sí	En parte	No	Evidencia o comentario
	Las preguntas bajo el umbral no llaman al modelo de generación				
La evaluación de resultados					
	<code>resultados.csv</code> tiene las seis preguntas con ROUGE-L, BLEU, coherencia y relevancia				
	Al recalcular una fila obtuve un valor igual o muy parecido				
En un entorno de desarrollo de software	<code>prompts.md</code> registra tres o más iteraciones con su efecto y las pruebas de alucinación, wording y falta de control; los ajustes y la recomendación se apoyan en los números				
	El README me bastó para instalar y ejecutar				
	No encontré claves en el código ni en el historial; <code>.env.example</code> no tiene valores				
	Los commits muestran el avance del trabajo, con mensajes que se entienden				

Cierre de la revisión

Escribe tres líneas, cada una con evidencia:

- **Una fortaleza:** qué hizo bien y dónde se ve.
- **Una mejora:** qué cambiarías, en qué componente y por qué.
- **Un siguiente paso:** una acción concreta que tu compañero/a puede hacer antes de publicar su portafolio.

Ejemplo. Fortaleza: tu `limpiar` reemplaza el correo de Q6 por `[correo]` y tiene una prueba que lo comprueba (`tests/test_preprocesar.py`). *Mejora:* en la evaluación de resultados, tu ROUGE-L de Q2 da 0,31 y a mí me da 0,62 con los mismos textos: parece que tokenizas la referencia con `split()` y la respuesta con `spaCy`, así que "XLSX." y "XLSX" o "Clientes," y "clientes" no cuentan como la misma palabra. *Siguiente paso:* usa la misma función de tokenización para las dos, recalcula la tabla y revisa si tu recomendación cambia.

Parte B · Trabajo en equipo en los ABPRO

Al cerrar cada ABPRO, valoras el trabajo de cada compañero/a de tu equipo. Los cuatro ABPRO del módulo son:

1. La arquitectura de la plataforma de soporte
2. Un cliente, dos proveedores
3. Torneo de clasificación de tickets
4. El control de calidad del resumidor

Completa una tabla como esta por cada ABPRO. Escala: 1 = no se observa; 2 = se observa a veces; 3 = se observa casi siempre; 4 = se observa siempre y el equipo se beneficia de ello.

Criterio	Compañero/a 1	Compañero/a 2	Compañero/a 3	Compañero/a 4
Nombre y rol en este ABPRO				
Cumple su rol (por ejemplo, el responsable de pruebas deja las pruebas funcionando sin conexión)				
Aporta ideas y soluciones				
Respeto los acuerdos y los plazos del equipo				
Se comunica con respeto, también cuando no está de acuerdo				
Ayuda a encontrar y corregir errores (por ejemplo, revisa un pull request o detecta un JSON inválido)				
Comentario (obligatorio si pusiste 1 o 2 en algún criterio)				

Qué hace el tutor/a con la coevaluación

- **Modera.** Revisa que cada comentario sea respetuoso y se apoye en evidencia; devuelve para corregir los que no cumplan, antes de que lleguen a su destinatario/a.
- **Valora tu retroalimentación, no el trabajo de tu compañero/a.** Tu revisión no cambia la nota de nadie. El tutor/a mira si tus comentarios son específicos (apuntan a un archivo, una línea o una fila), si se apoyan en lo que ejecutaste o recalculaste, si son accionables (dicen qué hacer) y si son respetuosos.
- **Consolida la Parte B.** Promedia los niveles que recibió cada integrante en cada criterio y le devuelve un resumen sin los nombres de quienes evaluaron. Si en un equipo aparecen diferencias fuertes, conversa con el equipo.
- **Cierra el ciclo.** Tú usas la coevaluación que recibiste en el cierre de tu portafolio: qué comentario recibiste y qué cambiaste por él.